Chapter 7 | Exercise solutions with Python

GPT, BERT, and T5

This notebook solves Exercises 6–11, including the two optional coding exercises. It follows the random decisions behind BERT corruption, locates the outputs that receive direct loss gradients, and tests which information can reach an output in each architecture. It also reproduces a fixed-example training run and compares two ways of adapting a causal stack with a classification head. Exercises 1–5 and 12 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements-ch05.txt`. Run the cells in order. The exercises train small models on the CPU and include their saved outputs.

```
from pathlib import Path
import sys

import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch07_pretraining" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )

torch.set_num_threads(1)
```

6. Follow all three BERT treatments

Start with *the red car stopped*. Python index 1 is the second position, which contains *red*. We select it by hand in every trial to isolate replacement from the earlier decision about which tokens to grade. Every call starts from the original sentence again, so a mask or random word from one trial never becomes the input to the next trial.

The helper saves the original ID as the label at the selected position. Then one uniform draw in $[0, 1)$ determines what the model sees. A draw below 0.8 gives the mask token. A draw from 0.8 up to 0.9 gives a random ordinary word, chosen by an additional integer draw. A draw of at least 0.9 keeps the original input. In all three branches the target is still *red*. The three other labels stay at the loss function's ignore value, `IGNORE` (−100).

This experimental copy of `bert_corrupt` in `examples/ch07_pretraining/data.py` prints the value already sampled by the helper, the branch that used it, and the resulting ID. The added print consumes no random numbers.

```
from contextlib import redirect_stdout
from io import StringIO

from examples.ch07_pretraining.data import (
    FIRST_WORD, IGNORE, MASK, VOCAB, _checked_selected,
    _checked_tokens, encode, token_names,
)


def traced_bert_corrupt(tokens, selected, generator):
    original = _checked_tokens(tokens)
    selected = _checked_selected(original, selected)
    inputs = original.clone()
    labels = torch.full_like(original, IGNORE)
    for index in selected:
        labels[index] = original[index]
        draw = torch.rand((), generator=generator).item()
        if draw < 0.8:
            inputs[index] = MASK
            treatment = "mask"
        elif draw < 0.9:
            inputs[index] = torch.randint(
                FIRST_WORD, len(VOCAB), (), generator=generator
            )
            treatment = "random"
        else:
            treatment = "unchanged"
        print(f"draw={draw:.9f}  {treatment:9s}  ID={inputs[index].item()}")
    return inputs, labels
```

Create the generator once, outside the loop. If we seeded a new generator with 19 on every call, each call would start at the same point in the random sequence and repeat the first treatment. Here the random branch advances the same generator a second time when it chooses a word. The trace records the actual draws in call order, including the effects of that additional draw.

We check the labels and the unselected inputs on every trial. We save every printed draw in memory and display the first example of each treatment and the first random replacement that reproduces *red*. Run `print(draw_log.getvalue())` in a cell to read the complete trace.

```
original = torch.tensor(encode("the red car stopped"))
selected_index = 1
original_id = original[selected_index]
unselected = torch.arange(original.numel()) != selected_index
corruption_rng = torch.Generator().manual_seed(19)

draw_log = StringIO()
first_examples = {}
same_word_example = None
for trial in range(1, 257):
    trace_start = draw_log.tell()
    with redirect_stdout(draw_log):
        inputs, labels = traced_bert_corrupt(
            original, [selected_index], corruption_rng
        )
    trace = draw_log.getvalue()[trace_start:].strip()
    treatment = trace.split()[1]

    assert labels[selected_index] == original_id
    assert torch.all(labels[unselected] == IGNORE)
    assert torch.equal(inputs[unselected], original[unselected])
    example = trial, trace, inputs.tolist(), labels.tolist()
    first_examples.setdefault(treatment, example)
    if treatment == "random" and inputs[selected_index] == original_id:
        same_word_example = example
    if len(first_examples) == 3 and same_word_example is not None:
        break

assert set(first_examples) == {"mask", "random", "unchanged"}
print("original IDs:", original.tolist())
examples_to_show = list(first_examples.values())
if same_word_example is not None:
    examples_to_show.append(same_word_example)
else:
    print("No same-word random replacement in 256 calls.")
for trial, trace, input_ids, label_ids in examples_to_show:
    print(f"call {trial:3d}  {trace}")
    print(f"  inputs {input_ids}  labels {label_ids}")
```

OUTPUT

```
original IDs: [7, 8, 9, 10]
call   1  draw=0.968602955  unchanged  ID=8
  inputs [7, 8, 9, 10]  labels [-100, 8, -100, -100]
call   2  draw=0.199866176  mask       ID=3
  inputs [7, 3, 9, 10]  labels [-100, 8, -100, -100]
call   3  draw=0.879927933  random     ID=7
  inputs [7, 7, 9, 10]  labels [-100, 8, -100, -100]
call  93  draw=0.872130215  random     ID=8
  inputs [7, 8, 9, 10]  labels [-100, 8, -100, -100]
```

In the recorded run, calls 1–3 cover all three treatments. The mask changes ID 8 to ID 3 (`[MASK]`), and the first random replacement changes ID 8 to ID 7 (*the*). Call 1 and call 93 both leave the input IDs at `[7, 8, 9, 10]` with labels `[-100, 8, -100, -100]`. Call 1 used the unchanged branch because its draw was about 0.969. Call 93 used the random branch because its draw was about 0.872, and the additional integer draw happened to return ID 8.

The replacement range includes all ten ordinary-word IDs from 7 through 16, including the original. Given the random branch in this vocabulary, drawing *red* has probability 1/10. The arrays alone cannot distinguish the two calls. The recorded draw identifies the branch, while the label records the original word in either case.

7. Find where the direct loss acts

Imagine a table with four rows, one per input position, and one column for each of the $V = 17$ vocabulary items. A **logit** is the score in a cell of that table before applying softmax. We make a batch with one table, so the tensor has shape $[1, 4, V]$. The only graded label is *red* at Python index 1. The other three entries are `IGNORE`, so `selected_loss` excludes their rows from both the sum of token losses and the count used to calculate the mean.

Write $z_{i,k}$ for the logit at position i for vocabulary ID k in this one-example batch. The target ID at position 1 is $y = 8$. With a single graded position, the loss is

$$L = -z_{1,y} + \log\left(\sum_{k=0}^{V-1} \exp z_{1,k}\right).$$

Only row 1 appears in this expression. Its gradient is the row's softmax probability vector minus a vector that is 1 at ID y and 0 everywhere else. To see why, differentiate the log of the sum. At ID k the derivative is $\exp z_{1,k}$ divided by the sum of all V exponentials. Differentiating $-z_{1,y}$ subtracts 1 only at the correct ID. Every other row is absent from L, so its partial derivatives are exactly zero.

```
from examples.ch07_pretraining.model import TinyTokenModel, selected_loss

vocab_size = len(VOCAB)
target_id = encode("red")[0]
loss_rng = torch.Generator().manual_seed(73)
random_logits = torch.randn(
    (1, 4, vocab_size), generator=loss_rng, requires_grad=True
)
loss_labels = torch.tensor([[IGNORE, target_id, IGNORE, IGNORE]])
direct_loss = selected_loss(random_logits, loss_labels)
direct_loss.backward()

direct_gradient = random_logits.grad[0]
expected_gradient = torch.zeros_like(direct_gradient)
expected_gradient[1] = random_logits.detach()[0, 1].softmax(dim=-1)
expected_gradient[1, target_id] -= 1
torch.testing.assert_close(direct_gradient, expected_gradient)
assert torch.count_nonzero(direct_gradient[[0, 2, 3]]) == 0

print(f"Loss: {direct_loss.item():.6f}"
      f"  |  correct vocabulary ID: {target_id}")
for index, row in enumerate(direct_gradient):
    print(f"Position {index} (nonzero entries: {row.count_nonzero()})")
    for block in row.split(7):
        print("  " + " ".join(f"{value:+.5f}" for value in block))
```

OUTPUT

```
Loss: 3.428197  |  correct vocabulary ID: 8
Position 0 (nonzero entries: 0)
  +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000
  +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000
  +0.00000 +0.00000 +0.00000
Position 1 (nonzero entries: 17)
  +0.08549 +0.11230 +0.04622 +0.05348 +0.06714 +0.02888 +0.20831
  +0.00961 -0.96755 +0.02028 +0.05529 +0.01710 +0.05030 +0.00971
  +0.13094 +0.03508 +0.03742
Position 2 (nonzero entries: 0)
  +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000
  +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000
  +0.00000 +0.00000 +0.00000
Position 3 (nonzero entries: 0)
  +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000
  +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000 +0.00000
  +0.00000 +0.00000 +0.00000
```

The numbers in each row follow vocabulary ID order from 0 to 16. In row 1, the ninth number belongs to ID 8 and is negative. A small increase to that logit reduces the loss. The other 16 numbers are positive because increasing a competing logit takes probability away from the correct ID. Rows 0, 2, and 3 have 17 exact zeros each.

Now feed *the [MASK] car stopped* to the bidirectional teaching model and keep the same labels. The loss still has a direct gradient only at the second output. That output can attend to *the*, *car*, and *stopped*. Each of those input vectors can affect the values or attention weights used to build the hidden vector at index 1. The chain rule propagates the loss along those paths back to the input vectors. The hook below saves the embedding lookup result so we can inspect gradients by input **position**, even if a token ID appears more than once in a sentence.

```
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(74)
    gradient_model = TinyTokenModel(causal=False).eval()

lookup_outputs = []

def save_embedding_output(module, arguments, output):
    output.retain_grad()
    lookup_outputs.append(output)

masked_ids = encode("the red car stopped")
masked_ids[1] = MASK
gradient_input = torch.tensor([masked_ids])
with gradient_model.embedding.register_forward_hook(save_embedding_output):
    model_logits = gradient_model(gradient_input)
    model_logits.retain_grad()
    selected_loss(model_logits, loss_labels).backward()

output_norms = model_logits.grad[0].norm(dim=-1)
input_norms = lookup_outputs[0].grad[0].norm(dim=-1)
assert torch.count_nonzero(output_norms[[0, 2, 3]]) == 0
assert torch.all(input_norms[[0, 2, 3]] > 0)
print(f"{'pos':>3}  {'input':7s}  {'logit grad norm':>16s}"
      f"  {'lookup grad norm':>16s}")
for index, name in enumerate(token_names(gradient_input[0])):
    print(f"{index:3d}  {name:7s}  {output_norms[index]:16.8f}"
          f"  {input_norms[index]:16.8f}")
```

```
OUTPUT
pos  input     logit grad norm  lookup grad norm
  0  the            0.00000000        0.08548550
  1  [MASK]         1.01604819        0.63174695
  2  car            0.00000000        0.19065119
  3  stopped        0.00000000        0.05139230
```

In this run the three ungraded positions have nonzero input-vector gradient norms, even though their output-gradient norms are zero. The lookup-vector gradient at a position comes from all graded outputs whose computations depend on it. When the model updates its embedding table, gradients from repeated uses of the same ID are added into that ID's shared row. An allowed path can carry a zero or small gradient for particular weights or inputs. The experiment exhibits a path that actually carries a signal here.

8. Test a future-token boundary

First run the book's `visibility_probe` in `examples/ch07_pretraining/run_examples.py`. It compares *the red car stopped* with *the red car moved* at Python index 1 (*red*). Then change the shared color to *blue* and repeat the comparison at the same index. The only difference **within each pair** is at index 3. The two stacks have the same parameters and buffers. They differ in their attention permission rule, so any difference in the measured sensitivity arises from that rule.

Compare the two whole vocabulary vectors **within** each stack. Our summary is the maximum absolute coordinate change over all V logits. With CPU `float64` computation, use an absolute tolerance of 10^{-12} and zero relative tolerance for the causal check. We also print the two changed-word logits as examples of individual coordinates in the comparison. Both models are in evaluation mode and this probe requires no gradients.

```
from examples.ch07_pretraining.run_examples import visibility_probe

tolerance = 1e-12
with torch.random.fork_rng(devices=[]):
    canonical = visibility_probe()
    torch.manual_seed(71)
    causal_stack = TinyTokenModel(causal=True).double().eval()
    bidirectional_stack = TinyTokenModel(causal=False).double().eval()
    bidirectional_stack.load_state_dict(causal_stack.state_dict())

print("Canonical pair at red")
for stack_name, difference in canonical.items():
    print(f"  {stack_name:13s} max |change| = {difference:.12g}")
assert canonical["causal"] <= tolerance

blue_before = torch.tensor([encode("the blue car stopped")])
blue_after = torch.tensor([encode("the blue car moved")])
blue_index = 1
report_ids = encode("stopped moved")
assert torch.where(blue_before[0] != blue_after[0])[0].tolist() == [3]
```

```
OUTPUT
Canonical pair at red
  causal        max |change| = 0
  bidirectional max |change| = 0.195011679533
```

```
print("New pair at blue")
for stack_name, stack in (
    ("causal", causal_stack), ("bidirectional", bidirectional_stack)
):
    with torch.no_grad():
        before = stack(blue_before)[0, blue_index]
        after = stack(blue_after)[0, blue_index]
    max_difference = (after - before).abs().max().item()
    print(f"  {stack_name:13s} max |change| = {max_difference:.12g}")
    for word in report_ids:
        print(f"    {VOCAB[word]:7s} {before[word]:+.8f}"
              f" -> {after[word]:+.8f}")
    if stack_name == "causal":
        torch.testing.assert_close(before, after, atol=tolerance, rtol=0)
```

OUTPUT

```
New pair at blue
  causal        max |change| = 0
    stopped -0.51536650 -> -0.51536650
    moved   -0.16878307 -> -0.16878307
  bidirectional max |change| = 0.168832161318
    stopped -0.45795604 -> -0.61841434
    moved   -0.01385238 -> -0.18268454
```

In both pairs the causal stack gives the same logits before and after the change at index 3. To see why this holds through multiple layers, start at the embeddings. The first two positions use the same token IDs and position vectors in the two inputs. At every causal layer a query at index i can read only keys at indices $j \leq i$. Thus no layer can move information from index 3 to index 1, directly or through an intermediate position. The residual connections, layer normalization, feed-forward network, and final projection act within each position, so they preserve this argument. The causal invariance follows from the mask and the other operations in this stack. The numeric tolerance only governs how we check the equality on a computer.

The bidirectional stack allows index 1 to read index 3, and the nonzero difference in each pair shows that the changed word reached the earlier logits with these weights. The same permission could yield a small or zero difference if a head gives that position very little weight, if the changed values project to the same features, or if later computations cancel the change. A nonzero difference is an observation for this particular probe.

9. Inspect a training run

From the repository root, run `python -m examples.ch07_pretraining.run_examples --train`. This invokes the three teaching models on one fixed training example per family for 200 Adam steps. We run the same module from the notebook using its Python interpreter and parse the printed JSON so that the table and token comparison come from the actual run.

`selected_labels` is the number of label slots graded in each fixed example. The reported NLL, or negative log-likelihood, is the mean of the token cross-entropies over those slots. In the BERT example only the masked *red* is graded. For GPT each of its three inputs supplies one next-token target. For T5 all eight decoder targets are graded, including the sentinels and `EOS`.

```
import json
import subprocess

command = [
    sys.executable, "-m", "examples.ch07_pretraining.run_examples",
    "--train",
]
run_result = subprocess.run(
    command, cwd=repo_root, check=True, capture_output=True, text=True
)
report = json.loads(run_result.stdout)
checks = report["fixed_example_checks"]
expected_counts = {"gpt": 3, "bert": 1, "t5": 8}

print(f"{'Family':6s} {'Graded':>6s} {'Initial NLL':>12s}"
      f" {'Final NLL':>12s} {'Correct':>8s}")
for family, graded in expected_counts.items():
    result = checks[family]
    predicted = result["selected_predictions"]
    targets = result["selected_targets"]
    correct = sum(
        pred == target for pred, target in zip(predicted, targets)
    )
    assert result["steps"] == 200
    assert result["selected_labels"] == graded == len(targets)
    assert len(predicted) == graded
    assert result["correct_selected"] == correct == graded
    assert result["final_nll"] < result["initial_nll"]
    print(f"{family.upper():6s} {graded:6d} {result['initial_nll']:12.8f}"
          f" {result['final_nll']:12.8f} {correct:4d}/{graded:<3d}")
```

OUTPUT

```
Family Graded  Initial NLL    Final NLL  Correct
GPT         3   2.98642468   0.00116479    3/3
BERT        1   2.88157463   0.00049924    1/1
T5          8   2.89228392   0.00153717    8/8
```

```
for family, result in checks.items():
    print(family.upper())
    print("  predictions", " ".join(result["selected_predictions"]))
    print("  targets    ", " ".join(result["selected_targets"]))
assert checks["t5"]["exact_generated"]
print("  generated  ", " ".join(checks["t5"]["generated"]))
```

```
OUTPUT
GPT
  predictions red car stopped
  targets     red car stopped
BERT
  predictions red
  targets     red
T5
  predictions S0 red car S1 old bridge S2 EOS
  targets     S0 red car S1 old bridge S2 EOS
  generated   S0 red car S1 old bridge S2 EOS
```

GPT predicts *red*, *car*, and *stopped* in order, BERT predicts *red* at the mask, and T5 matches all eight tokens. For the T5 selected predictions, the decoder received the correct shifted target during the forward pass, a setup called **teacher forcing**. The run also reports a separate greedy generation check. It starts at `BOS`, feeds back each token it predicts, and reproduces the target through `EOS` for this same training source.

The loss drops and the targets match after optimization. Together these results check that the target shifts, masking, vocabulary projection, loss, gradient, and optimizer work together well enough to fit these three training examples. Repeating one fixed example lets a model memorize its targets. The run gives no held-out measurement, and the three means summarize different targets and different visible information. For example, BERT scores one original word with left and right context, while GPT scores three words using only their prefixes. A smaller training mean on one of these objectives does not imply better predictions on a common task.

For an initial held-out check, reserve *the blue car moved near the old bridge today*. Use its first four words for the two short objectives and the whole sentence for T5, mirroring the lengths used in the training demo. Reserve both forms before generating corruptions so none of their derived inputs is used for training. Keep the weights fixed during evaluation and use these examples.

Family	Held-out input	Graded target
GPT	`the blue car`	`blue car moved`
BERT	`the [MASK] car moved`	`blue` at index `1`
T5	`the S0 moved near the S1 today EOS`	`S0 blue car S1 old bridge S2 EOS`

For T5 the teacher-forced decoder input is `BOS S0 blue car S1 old bridge S2`. Score the graded slots of each family on its own objective and separately check T5's freely generated sequence for an exact match. This is a new in-vocabulary sentence, although *blue* and *moved* did not appear in the demo's training examples. Its score could reflect that lack of training exposure as well as the change of sentence. To isolate recombination in a larger experiment, expose the model to those words in several other training sentences while holding back their new combinations. Because continuations and masked words can be ambiguous, evaluate many held-out sentences and use the recorded targets rather than treating a single answer as the only grammatical possibility.

10. Check both sides of an encoder–decoder (optional)

Exercise. Use one small random encoder–decoder in evaluation mode. Compare decoder logits after changing a later decoder input, replacing a valid source token, and appending a masked source PAD. Write out the source validity masks.

Use the same span-corruption example as Exercise 4. Its decoder input begins with `BOS S0 red car S1 old`. At decoder index 1, the input is `S0` and the model predicts the next token, `red`. The causal self-attention mask lets that prediction use decoder inputs at indices 0 and 1. Cross-attention can also use every valid source key, including the source token `stopped` at index 2. Decoder index 1 and source index 2 belong to different sequences, so their index order places no restriction on cross-attention.

Keep one set of weights for all calls. Use CPU float64, evaluation mode, and disabled gradient recording. Compare whole vectors of vocabulary logits using the maximum absolute difference. With absolute tolerance `1e-12` and zero relative tolerance, two slices pass the invariance check if each corresponding entry differs by at most `1e-12`.

```
from examples.ch07_pretraining.data import (
    PAD, TOKEN_TO_ID, encode, span_corrupt, token_names,
)
from examples.ch07_pretraining.model import make_model

torch.manual_seed(310)
t5_probe = make_model("t5", n_layers=2).double().eval()

source_ids, decoder_ids, _ = span_corrupt(
    encode("the red car stopped near the old bridge today"),
    [(1, 3), (6, 8)],
)
source_ids = source_ids[None]
decoder_ids = decoder_ids[None]

valid_source = torch.ones_like(source_ids, dtype=torch.bool)
valid_decoder = torch.ones_like(decoder_ids, dtype=torch.bool)
padded_source = F.pad(source_ids, (0, 1), value=PAD)
valid_padded_source = F.pad(valid_source, (0, 1), value=False)

print("Source tokens:", token_names(source_ids[0]))
print("Decoder input:", token_names(decoder_ids[0]))
print("Unpadded mask:", valid_source.tolist())
print("Padded mask:", valid_padded_source.tolist())
```

OUTPUT

```
Source tokens: ['the', 'S0', 'stopped', 'near', 'the', 'S1', 'today', 'EOS']
Decoder input: ['BOS', 'S0', 'red', 'car', 'S1', 'old', 'bridge', 'S2']
Unpadded mask: [[True, True, True, True, True, True, True, True]]
Padded mask: [[True, True, True, True, True, True, True, True, False]]
```

Replace `old` with `blue` at decoder index 5 and measure the logits at indices 0–4. At every decoder layer, each of those earlier positions reads only its own prefix, so there is no route from input index 5 to the earlier logits. Decoder row 5 can read its own input and provides a useful check that the replacement reached the model.

Next, change `stopped` to `moved` at source index 2 and hold the entire decoder input fixed. The source encoder can incorporate the replacement into any valid source row, and a decoder query can read the resulting source memory. Even logits at decoder index 0, where the decoder input is just `BOS` at that position, may change.

Finally, append a PAD on the right, preserving the original source indices. The `False` mask entry excludes the new position as a key in encoder self-attention and decoder cross-attention. Compare every decoder output, since the source padding restriction applies to every decoder query.

```
later_position = 5
changed_decoder = decoder_ids.clone()
changed_decoder[0, later_position] = TOKEN_TO_ID["blue"]

changed_source = source_ids.clone()
changed_source[0, 2] = TOKEN_TO_ID["moved"]

with torch.inference_mode():
    base_logits = t5_probe(
        source_ids, decoder_ids, valid_source, valid_decoder
    )
    later_logits = t5_probe(
        source_ids, changed_decoder, valid_source, valid_decoder
    )
    source_logits = t5_probe(
        changed_source, decoder_ids, valid_source, valid_decoder
    )
    pad_logits, pad_weights = t5_probe(
        padded_source, decoder_ids, valid_padded_source,
        valid_decoder, return_weights=True,
    )
```

```
def max_logit_delta(first, second):
    return (first - second).abs().max().item()


early = slice(0, later_position)
cases = [
    ("Later decoder edit, rows 0–4",
     base_logits[:, early], later_logits[:, early]),
    ("Later decoder edit, row 5",
     base_logits[:, 5], later_logits[:, 5]),
    ("Valid source edit, decoder row 0",
     base_logits[:, 0], source_logits[:, 0]),
    ("Masked source PAD, all decoder rows",
     base_logits, pad_logits),
]
atol = 1e-12
for description, first, second in cases:
    print(f"{description:<36} {max_logit_delta(first, second):.3e}")

torch.testing.assert_close(
    base_logits[:, early], later_logits[:, early],
    atol=atol, rtol=0,
)
torch.testing.assert_close(base_logits, pad_logits, atol=atol, rtol=0)
for attention_path in ("encoder", "cross"):
    assert all(
        layer[..., -1].count_nonzero().item() == 0
        for layer in pad_weights[attention_path]
    )
    print(f"{attention_path}: PAD key weight is zero in every layer")
```

OUTPUT

```
Later decoder edit, rows 0–4         0.000e+00
Later decoder edit, row 5            1.210e+00
Valid source edit, decoder row 0     6.912e-02
Masked source PAD, all decoder rows  0.000e+00
encoder: PAD key weight is zero in every layer
cross: PAD key weight is zero in every layer
```

The saved run reports zero difference for decoder rows 0–4 after the later decoder edit. At the edited row 5, at least one logit changes by about 1.21. Replacing the valid source token changes a row-0 logit by about 0.0691. The mask guarantees the earlier-row invariance. In the two other cases, the path is available and the size of the difference depends on the weights and inputs.

The masked PAD has zero key weight in both attention paths. The extra source position can still act as a query and produce a hidden vector. That vector cannot influence valid source rows or decoder rows because the key mask excludes it at each attention layer. Appending PAD leaves the original sinusoidal position indices unchanged. Floating-point kernels can round differently after a shape change, which is why the logit comparison uses a tolerance.

11. Add a task head (optional)

Exercise. Adapt the causal token model to classify a sentence as `red` (class 0) or `blue` (class 1). Compare training just a new classification head with updating the entire classifier. Use the same starting weights in both conditions and reserve a test split for the selected models.

Start from a color-free sentence template such as "today the {color} car stopped." Substituting `red` and `blue` produces a *counterfactual pair*: only the color word changes. Give every template both labels and assign each entire pair to one split. This balances sentence length, the color's position, and all non-color words across labels within each split. Validation and test use sentence arrangements held out from training. The color words themselves appear in training because the task is to recognize those words in new arrangements.

```
from copy import deepcopy

from examples.ch07_pretraining.data import EOS
from examples.ch07_pretraining.model import TinyTokenModel

train_templates = (
    "the {color} car stopped",
    "the {color} car moved",
    "today the {color} car stopped",
    "today the {color} car moved",
    "the {color} car stopped today",
    "the {color} car moved today",
    "the {color} car stopped near the bridge",
    "the {color} car moved near the bridge",
    "the car stopped near the {color} bridge",
    "the car moved near the {color} bridge",
    "near the {color} bridge the car stopped",
    "near the {color} bridge the car moved",
)
validation_templates = (
    "the {color} car stopped near the old bridge",
    "today the {color} car moved near the bridge",
    "the car stopped near the {color} bridge today",
    "near the old {color} bridge the car moved",
)
test_templates = (
    "the old {color} car moved",
    "today the car stopped near the {color} bridge",
    "the {color} car moved near the old bridge today",
    "near the {color} bridge today the car stopped",
    "the car moved near the old {color} bridge",
    "near the old bridge the {color} car stopped",
)

split_templates = (train_templates, validation_templates,
                   test_templates)
all_templates = sum(split_templates, ())
assert len(set(all_templates)) == len(all_templates)
assert all(text.count("{color}") == 1 for text in all_templates)
```

Append `EOS` after the sentence, then right-pad the sequences within each batch. The colors stay in the input as ordinary vocabulary IDs. The supervised target is a separate class index per sentence. It never enters the token stack. Because each pair uses an identical template, a rule based only on words such as `car` or `bridge` must get exactly one of the two labels right.

```
class_names = ("red", "blue")
color_ids = {TOKEN_TO_ID[name] for name in class_names}


def color_batch(templates):
    sentences, rows, labels = [], [], []
    for template in templates:
        for label, color in enumerate(class_names):
            sentence = template.format(color=color)
            row = encode(sentence) + [EOS]
            assert sum(token in color_ids for token in row) == 1
            sentences.append(sentence)
            rows.append(torch.tensor(row))
            labels.append(label)
    ids = nn.utils.rnn.pad_sequence(
        rows, batch_first=True, padding_value=PAD
    )
    return ids, torch.tensor(labels), sentences


train_ids, train_labels, _ = color_batch(train_templates)
valid_ids, valid_labels, _ = color_batch(validation_templates)
print("Training class counts:", train_labels.bincount().tolist())
print("Validation class counts:", valid_labels.bincount().tolist())
```

```
OUTPUT
Training class counts: [12, 12]
Validation class counts: [4, 4]
```

The chapter's token model already computes a final vector of length d_{model} at every position. Its usual vocabulary projection maps that vector to 17 token scores. Replace the projection with `nn.Identity()`, which returns its input unchanged. This makes the same forward pass return hidden vectors with shape $[B, T, d_{model}]$ while retaining the embeddings, positions, causal mask, and layers. Here B is the number of sentences in the batch and T is the number of positions in each padded row.

For a row with L valid positions, the last valid index is $L - 1$ because positions start at zero and padding is on the right. That index contains the appended `EOS`. Its causal prefix includes the whole sentence, even when the color occurs near the end. Index each row by its own length so that pooling never selects PAD. A linear map on the resulting vector outputs two scores per sentence. Cross-entropy uses those logits and the integer class index, and the predicted class is the index of the larger logit.

```
class ColorClassifier(nn.Module):
    def __init__(self, backbone):
        super().__init__()
        self.backbone = backbone
        self.backbone.vocabulary_projection = nn.Identity()
        self.head = nn.Linear(backbone.d_model, len(class_names))

    def forward(self, ids):
        hidden = self.backbone(ids)
        last_valid = (ids != PAD).sum(dim=1) - 1
        batch_indices = torch.arange(ids.shape[0], device=ids.device)
        sentence_vectors = hidden[batch_indices, last_valid]
        return self.head(sentence_vectors)


torch.manual_seed(311)
color_base = ColorClassifier(TinyTokenModel(causal=True))
example_logits = color_base(train_ids[:2])
print("One two-logit row per sentence:", list(example_logits.shape))
print("Trainable parameters if the stack is frozen:",
      sum(p.numel() for p in color_base.head.parameters()))
print("Trainable parameters if the stack is updated:",
      sum(p.numel() for p in color_base.parameters()))
```

OUTPUT

```
One two-logit row per sentence: [2, 2]
Trainable parameters if the stack is frozen: 34
Trainable parameters if the stack is updated: 4626
```

For each training condition and learning rate, copy `color_base`, including its randomly initialized head. A deep copy gives each trial its own parameter tensors. Freeze the backbone in the head-only trials and pass only parameters that require gradients to a fresh Adam optimizer. In the full-network trials the trainable backbone can change how it encodes the sentence. The classifier always updates its head.

Use the same full training batch in every update and run each candidate for 80 updates. Try learning rates 0.003 and 0.01 under the same rules. Record validation mean cross-entropy (negative log likelihood, or NLL) at initialization and after every 10 updates. Keep the lowest validation NLL checkpoint for each rate, breaking exact ties in favor of the earlier update. Then select the rate with the lowest validation NLL, breaking rate ties in favor of the smaller rate. Thus the budget in data exposure and updates matches across conditions. Backpropagating through the whole backbone costs more computation per update.

```
def fit_color_trial(freeze, learning_rate):
    model = deepcopy(color_base)
    model.backbone.requires_grad_(not freeze)
    optimizer = torch.optim.Adam(
        [p for p in model.parameters() if p.requires_grad],
        lr=learning_rate,
    )
    best = None
    for step in range(81):
        if step > 0:
            model.train()
            optimizer.zero_grad(set_to_none=True)
            loss = F.cross_entropy(model(train_ids), train_labels)
            loss.backward()
            optimizer.step()
        if step % 10 == 0:
            model.eval()
            with torch.no_grad():
                validation_nll = F.cross_entropy(
                    model(valid_ids), valid_labels
                ).item()
            if best is None or validation_nll < best[0]:
                best = validation_nll, step, deepcopy(model)
    return best


selected_trials = {}
for mode, freeze in (("head only", True), ("whole network", False)):
    candidates = []
    for learning_rate in (0.003, 0.01):
        val_nll, step, model = fit_color_trial(freeze, learning_rate)
        candidates.append((val_nll, learning_rate, step, model))
        print(f"{mode:<14} lr={learning_rate:.3f}  step={step:>2}  "
              f"validation NLL={val_nll:.6f}")
    selected_trials[mode] = min(
        candidates, key=lambda result: (result[0], result[1])
    )
```

OUTPUT

```
head only      lr=0.003  step=80  validation NLL=0.675192
head only      lr=0.010  step=30  validation NLL=0.674488
whole network  lr=0.003  step=80  validation NLL=0.002391
whole network  lr=0.010  step=80  validation NLL=0.000779
```

Check that freezing worked by measuring the largest change in any backbone parameter from initialization to the selected checkpoint. Then evaluate both selected models on the same held-out test batch. Report the number of correctly classified sentences and the number of pairs in which *both* colors are classified correctly. Pair accuracy exposes a model that gives the same answer when we swap the color. A fixed-class predictor scores half of the sentences and zero complete pairs.

```
test_ids, test_labels, test_sentences = color_batch(test_templates)
print("Test class counts:", test_labels.bincount().tolist())

for mode, (_, rate, step, model) in selected_trials.items():
    backbone_delta = max(
        (now.detach() - start.detach()).abs().max().item()
        for now, start in zip(model.backbone.parameters(),
                              color_base.backbone.parameters())
    )
    if mode == "head only":
        assert backbone_delta == 0.0

    model.eval()
    with torch.inference_mode():
        logits = model(test_ids)
        predictions = logits.argmax(dim=-1)
        test_nll = F.cross_entropy(logits, test_labels).item()
    correct = predictions == test_labels
    complete_pairs = correct.reshape(-1, 2).all(dim=1)

    print(f"\n{mode}: lr={rate:.3f}, step={step}")
    print(f"  Maximum backbone change: {backbone_delta:.3e}")
    print(f"  Test NLL: {test_nll:.6f}")
    n_correct = correct.sum().item()
    print(f"  Correct sentences: {n_correct}/{len(correct)}")
    print(f"  Complete pairs: {complete_pairs.sum().item()}/"
          f"{len(complete_pairs)}")
    for sentence, actual, predicted in zip(
        test_sentences[:2], test_labels[:2], predictions[:2]
    ):
        print(f"  {sentence}")
        print(f"    target={class_names[actual]}, "
              f"prediction={class_names[predicted]}")
```

OUTPUT

```
Test class counts: [6, 6]

head only: lr=0.010, step=30
  Maximum backbone change: 0.000e+00
  Test NLL: 0.665279
  Correct sentences: 7/12
  Complete pairs: 1/6
  the old red car moved
    target=red, prediction=red
  the old blue car moved
    target=blue, prediction=blue

whole network: lr=0.010, step=80
  Maximum backbone change: 3.558e-01
  Test NLL: 0.000778
  Correct sentences: 12/12
  Complete pairs: 6/6
  the old red car moved
    target=red, prediction=red
  the old blue car moved
    target=blue, prediction=blue
```

Validation picks a learning rate of 0.01 in both conditions, but the frozen model's best checkpoint occurs at step 30 and the full-network model's best occurs at step 80. On the test split, the frozen model scores 7 out of 12 sentences and gets both members right in 1 of the 6 pairs. The full-network model scores 12 out of 12 sentences and gets all 6 pairs right. The two metrics describe different things. Getting 7 sentences right is one more success than always predicting a single label. Getting a whole pair right requires the model to respond correctly to a change in the color word while every other word stays fixed.

Both backbones start from random weights in this run. A frozen random backbone can already preserve some information about token identity through its embeddings and attention. Its head can learn a linear rule only from the features that reach the pooled `EOS` vector. Updating the backbone also lets the supervised signal change those features. With one seed and six test pairs, the observed difference describes this toy task and these settings.

To study **transfer**, first train a `TinyTokenModel` with the next-token objective on separate unlabeled text. Exclude all downstream sentence templates from that corpus and select the pretraining checkpoint with a separate language-model validation split. Load that checkpoint into a single backbone before replacing the vocabulary projection with the identity and initializing the two-logit task head. Copy that classifier for both adaptation regimes, give each trial a fresh optimizer, and run the same downstream validation and test protocol. Including the two random-start conditions alongside those two pretrained conditions lets you estimate the effect of pretraining at each level of adaptation.